

DBMS relazionali: caratteristiche evolute

Daniele Riboni

Università degli Studi di Cagliari

Dipartimento di Matematica e Informatica

Argomenti

- Transazioni e proprietà ACID
- Basi di dati attive e trigger
- Autorizzazioni e controllo degli accessi

TRANSAZIONE



Sequenza di operazioni da considerare **indivisibile** ("atomica"), **corretta** anche in presenza di concorrenza e con **effetti definitivi**

Esempio di transazione

Un bonifico di 100 euro dal conto di Tim al conto di Tom:

- Inizio_transazione;
- Sottrai 100 euro dal saldo del conto di Tim
- Aggiungi 100 euro al saldo del conto di Tom
- Fine transazione

Transazione

- **Proprietà ACID** ("acide"):
 - **A**tomicità
 - **C**onsistenza
 - **I**solamento
 - **D**urabilità (persistenza)

Atomicità

- La sequenza di operazioni sulla base di dati viene eseguita **per intero o per niente**:
 - Trasferimento di fondi da un conto A ad un conto B:
 - o si fanno il prelievo da A e il versamento su B
 - o nessuno dei due

Consistenza

- **Al termine** dell'esecuzione di una transazione, i vincoli di integrità devono essere soddisfatti
- "Durante" l'esecuzione ci possono essere violazioni, ma se restano alla fine allora la transazione deve essere annullata per intero

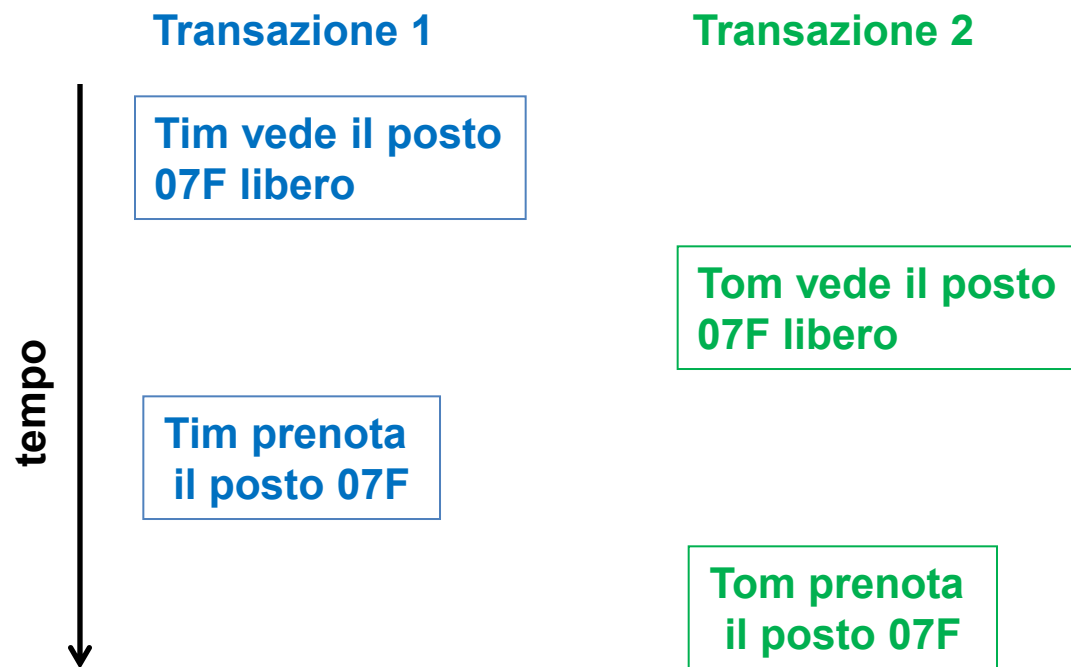
Isolamento

- L'effetto di **transazioni concorrenti** deve essere coerente (**equivalente all'esecuzione seriale**)
 - Se due assegni emessi sullo stesso conto corrente vengono incassati contemporaneamente si deve evitare di trascurarne uno

Isolamento

Le transazioni vengono eseguite in parallelo!

- Es.: Prenotazione di un posto per il volo AZ111



Durabilità (persistenza)

- La conclusione positiva di una transazione corrisponde ad un impegno (**commit**) a mantenere traccia del risultato in modo definitivo, anche in presenza di guasti

Transazioni in SQL

- Una transazione inizia al primo comando SQL dopo la "connessione" di un utente alla base di dati oppure alla conclusione di una precedente transazione
- (lo standard indica anche un comando **start transaction**, non obbligatorio, e quindi non previsto in molti sistemi)

Transazioni in SQL

- Conclusione di una transazione
 - **commit**: le operazioni specificate a partire dall'inizio della transazione vengono eseguite sulla base di dati,

oppure

- **rollback**: si rinuncia all'esecuzione delle operazioni specificate dopo l'inizio della transazione

Una transazione in SQL

```
start transaction;  
update ContoCorrente  
  set Saldo = Saldo - 10  
  where NumeroConto = 123;  
update ContoCorrente  
  set Saldo = Saldo + 10  
  where NumeroConto = 456;  
commit work;
```

Supporto proprietà ACID

- I DBMS relazionali **garantiscono** le proprietà **ACID**
- Ma garantirle non è semplice: ha un **impatto negativo sull'efficienza** del DBMS
- Nuovi DBMS non-relazionali (**NoSQL**) supportano **versioni deboli** di ACIDità per essere più efficienti
- DBMS relazionali di nuova generazione (**NewSQL**) cercano di **conciliare ACID ed efficienza**

BASI DI DATI ATTIVE



- Una base di dati che contiene regole attive (chiamate **trigger**)
- Paradigma: Evento-Condizione-Azione
 - Quando un evento si verifica
 - Se la condizione è vera
 - Allora l'azione è eseguita

Evento-Condizione-Azione

- **Evento** (normalmente una modifica dello stato del database: insert, delete, update)
 - Quando accade l'evento, il trigger è attivato
- **Condizione**
 - Un predicato che identifica se l'azione del trigger deve essere eseguita
- **Azione**
 - Una sequenza di comandi SQL

Il concetto di trigger

- Questo modello consente computazioni **reattive**
- Non è il solo tipo di regole:
 - Azioni compensative nei vincoli di integrità
- Vantaggio (rispetto ad implementare le regole esternamente):
 - **La logica dei dati è separata dalla logica delle applicazioni**

Sintassi SQL

- Ogni trigger è caratterizzato da:
 - Nome
 - Target (tabella controllata)
 - Modalità (before o after)
 - Evento (insert, delete o update)
 - Granularità (statement-level o row-level)
 - Azione
 - Timestamp di creazione

La sintassi standard non è supportata da molti DBMS

Sintassi SQL

```
create trigger NomeTrigger  
  { before | after }  
  { insert | delete | update [of Column] } on  
  TabellaTarget  
  [referencing  
    {[old table [as] VarTuplaOld  
      [new table [as] VarTuplaNew] } |  
    {[old [row] [as] VarTabellaOld  
      [new [row] [as] VarTabellaNew] }]  
  [for each { row | statement }]  
  [when Condizione]  
StatementProceduraleSQL
```

Granularità degli eventi

- Modalità **statement-level** (di default)
 - Il trigger viene considerato e possibilmente eseguito solo una volta per ogni statement (comando) che lo ha attivato, indipendentemente dal numero di tuple modificate
 - In linea con SQL (set-oriented)

Granularità degli eventi

- Modalità **row-level**
 - Il trigger viene considerato e possibilmente eseguito una volta per ogni tupla modificata

Variabili e granularità

- Se granularità = **row-level**
 - **old**: variabile che fa riferimento alla tupla nello stato precedente la modifica
 - **new**: variabile che fa riferimento alla tupla nello stato prodotto dalla modifica
- Se granularità = **statement-level**
 - **old_table**: stato precedente della porzione di tabella interessata dalla modifica
 - **new_table**: stato prodotto dalla modifica

Modalità

- BEFORE

- Il trigger è considerato e possibilmente eseguito prima della modifica
- I trigger before non possono modificare lo stato del database; possono al più condizionare i valori “new”
- Normalmente questa modalità è usata quando si vuole verificare una modifica prima che essa avvenga e “modificare la modifica”

Esempio “before”

- 1. “Conditioner” (agisce prima dell’update e della verifica di integrità)

```
create trigger LimitaAumenti  
before update of Salario on Impiegato  
for each row  
when (New.Salario > Old.Salario * 1.2)  
set New.Salario = Old.Salario * 1.2
```


Modalità

- **AFTER**
 - Il trigger è considerato e eseguito dopo l'evento
 - E' la modalità più comune

Esempio “after”

- 2. “Re-installer” (agisce dopo l’update)

```
create trigger LimitaAumenti  
after update of Salario on Impiegato  
for each row  
when(New.Salario > Old.Salario * 1.2)  
update Impiegato  
set New.Salario = Old.Salario * 1.2  
where ImpNum = new.ImpNum
```

Esempio di trigger row-level

```
create trigger AccountMonitor
after update on Account
for each row
when new.Totale < old.Totale
insert values
    (new.NumeroConto,
     old.Totale-new.Totale, now())
into Prelievi
```

Esempio di trigger statement-level

```
create trigger
ArchiviaFattureCancellate
after delete on Fattura
insert into FattureCancellate
(select *
 from old_table)
```

Esecuzione di Trigger in conflitto

- Quando vi sono più trigger associati allo stesso evento (in conflitto) vengono eseguiti come segue:
 - Per primi i **before** triggers
 - Poi viene eseguita la modifica
 - Infine sono eseguiti gli **after** triggers
- Quando vari trigger appartengono alla stessa categoria, l'ordine di esecuzione è definito in base al loro timestamp di creazione
 - I trigger più vecchi hanno priorità più alta

Terminazione

- L'azione di un trigger può produrre eventi che attivano altri trigger: si possono generare dei cicli infiniti
- Possibili esiti:
 - L'esecuzione termina correttamente
 - L'esecuzione termina in errore quando si raggiunge una data profondità di ciclo
 - Viene fatto un rollback: si ripristina lo stato precedente l'attivazione dei trigger

Analisi della terminazione

- Si usa una rappresentazione delle regole detta **grafo di triggering**:
 - Un nodo per ogni trigger
 - Un arco dal nodo t_i al nodo t_j se l'esecuzione dell'azione di t_i può attivare il trigger t_j
- Se il grafo è aciclico, l'esecuzione termina sempre
- Se il grafo ha cicli, esso **può** avere problemi di terminazione

Esempio con due trigger

T1:

```
create trigger AggiornaContributi
  after update of Stipendio on Impiegato
  update Impiegato
  set Contributi = Stipendio * 0.8
  where Matr in
    (select Matr from new_table)
```

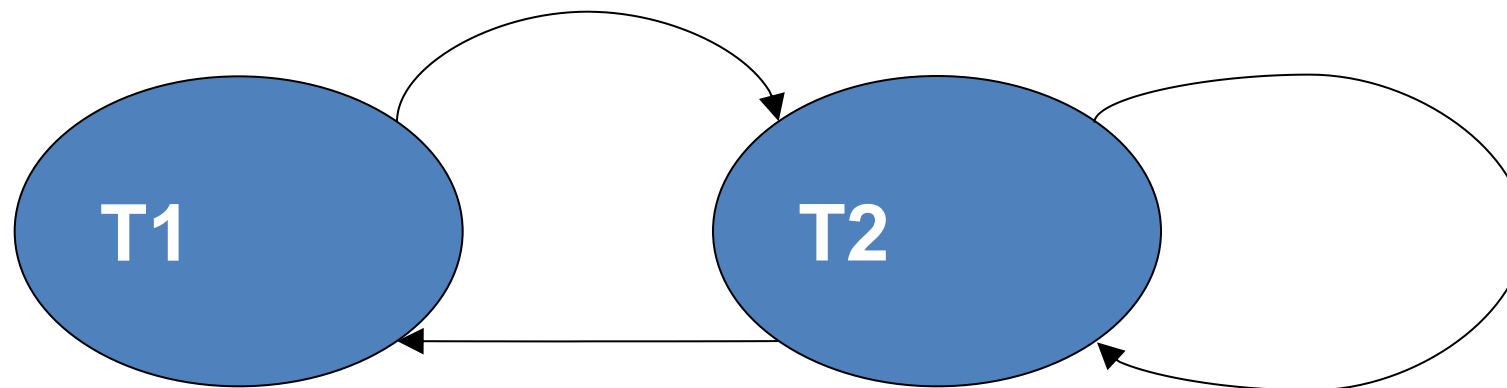
(mi permette di garantire un vincolo di
tupla)

Esempio con due trigger

T2:

```
create trigger ControllaSogliaBudget
after update on Impiegato
when 50000 < (select
    (New.Stipendio + New.Contributi)
    from Impiegato)
update Impiegato
set Stipendio = 0.9 * Stipendio
```

Grafo di triggering corrispondente

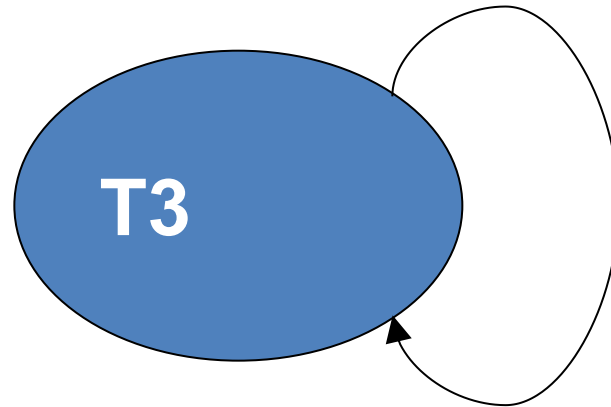


- Ci sono due cicli ma il sistema termina sempre
- Sarebbe non terminante se in T2 il fattore di moltiplicazione fosse maggiore di 1

Esempio di non terminazione

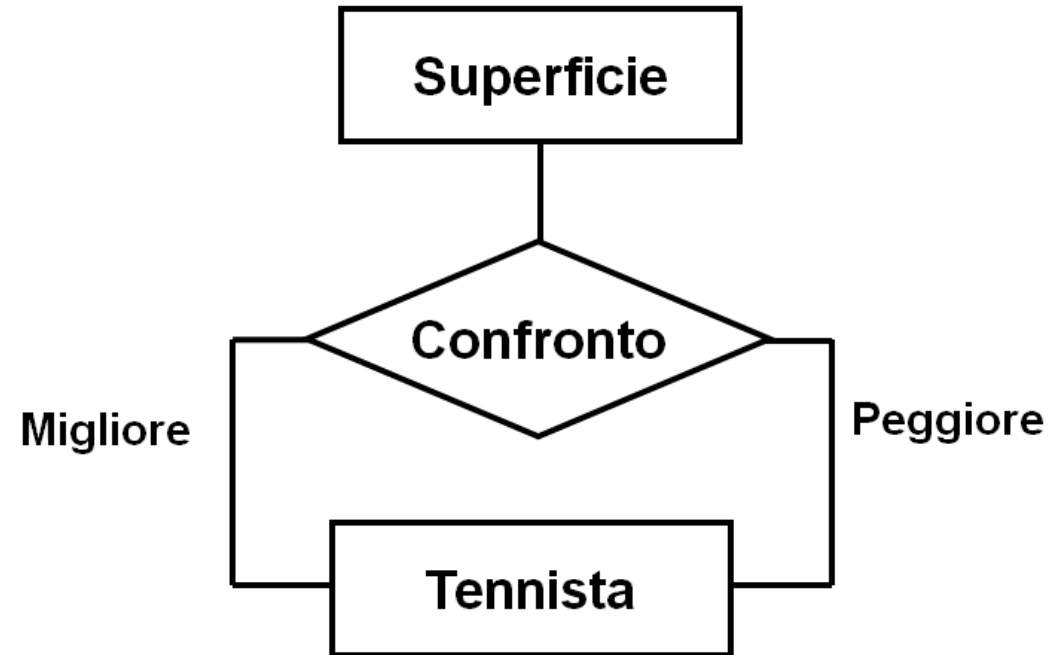
T3:

```
create trigger ControllaStipendio  
after update on Impiegato  
when New.Stipendio < 50000  
update Stipendio  
set Stipendio = 0.9 * Stipendio
```

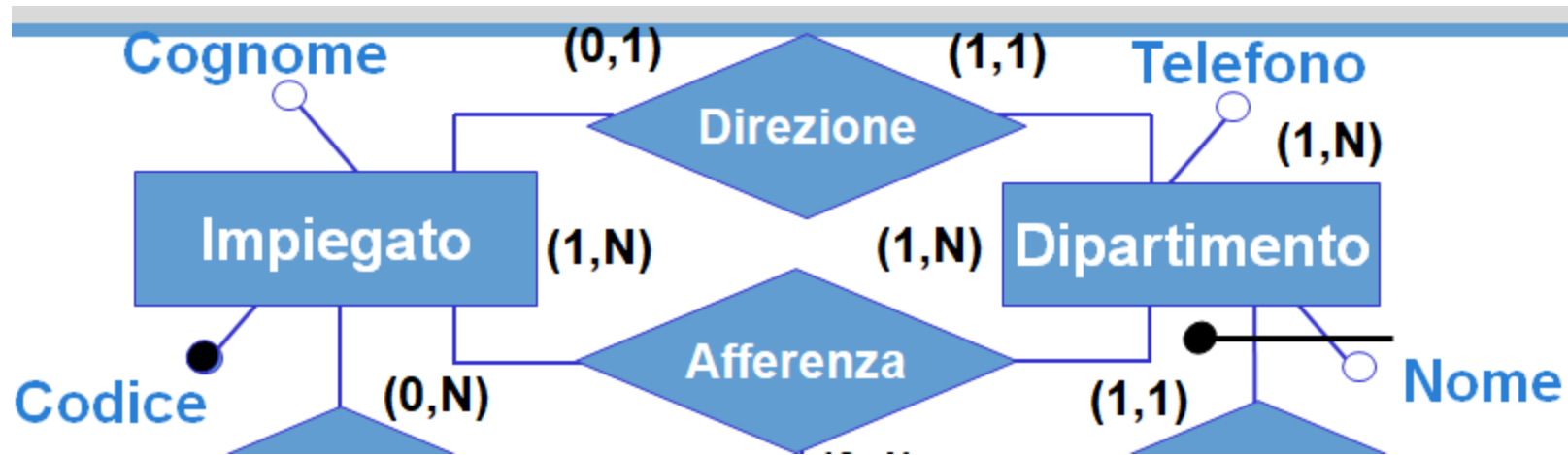


Trigger e controllo di consistenza

- Quali altri vincoli “non esprimibili in ER” potrei implementare tramite trigger?
- (L’associazione Confronto è asimmetrica) -
Dopo avere inserito la tupla “A migliore di B su S”
controllo: se esiste un’altra tupla “B migliore di A su S”, allora cancello la nuova tupla



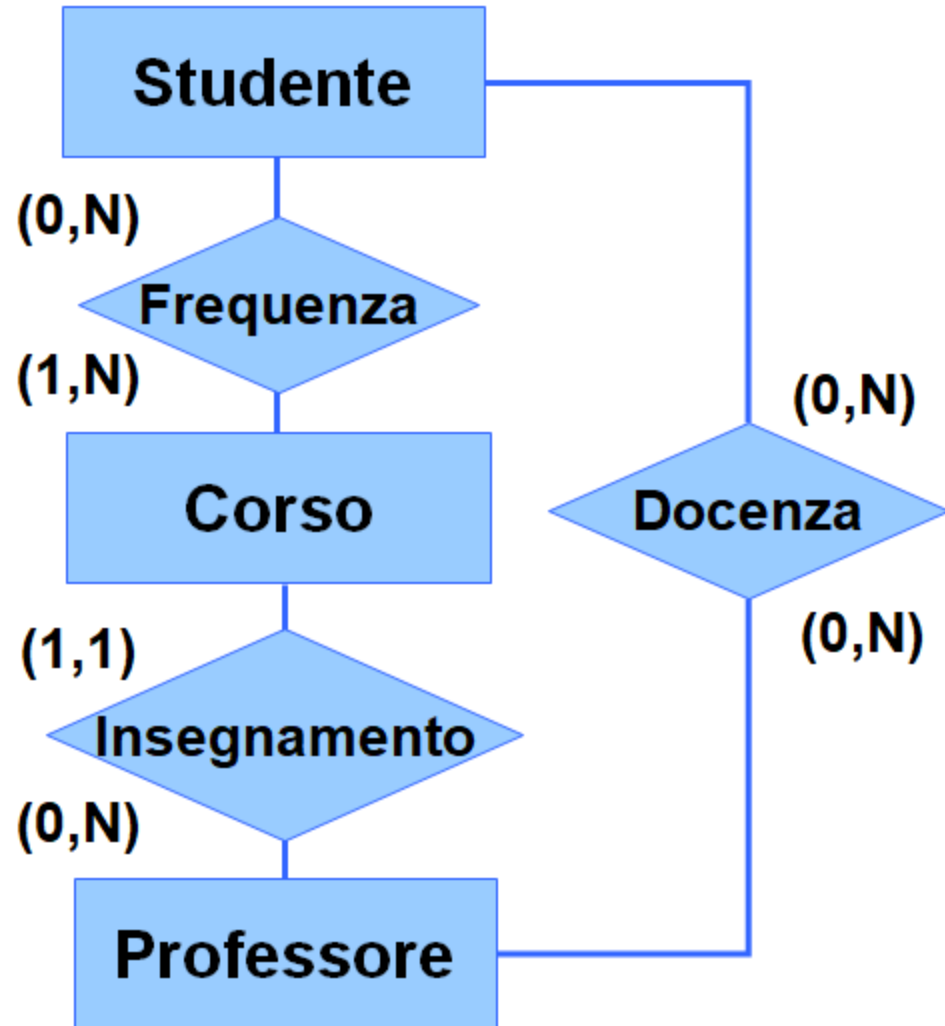
Trigger e controllo di consistenza



- (il direttore di un dipartimento deve afferire a quel dipartimento) - Dopo avere modificato o inserito la tupla “Il Dip. X ha come direttore l’impiegato Y”, controllo: se non ho una tupla (Y, X) nella relazione **Afferenza**, allora la aggiungo

Trigger e controllo di consistenza

- Dopo avere modificato o inserito
Frequenza(Stud1,C1),
seleziono il valore di Prof
dalla tupla della tabella Corso
di C1 (immaginiamo il prof
sia Prof1), e verifico se esiste
una tupla
Docenza(Prof1, Stud1); se la
tupla non esiste allora la
aggiungo



Discussione

- A cosa servono i trigger? Posso fare tutto da applicazione!
- Ma i trigger servono per implementare la logica dei dati, non delle applicazioni
- Permettono di implementare delle caratteristiche “invarianti” dei dati, che devono valere qualunque applicazione acceda ai dati

CONTROLLO DELL'ACCESSO



In SQL è possibile specificare chi (utente) e come (lettura, scrittura, ...) può utilizzare la base di dati (o parte di essa)

Controllo dell'accesso

- Oggetto dei **privilegi** sono di solito le tabelle, ma anche altri tipi di **risorse**, quali singoli attributi, viste o domini
- Un utente predefinito **_system** (amministratore della base di dati) ha tutti i privilegi
- Il creatore di una risorsa ha tutti i privilegi su di essa

Privilegi

- Un privilegio è caratterizzato da:
 - La risorsa cui si riferisce
 - L'utente che concede il privilegio
 - L'utente che riceve il privilegio
 - L'azione che viene permessa
 - La trasmissibilità del privilegio

Tipi di privilegi offerti da SQL

- **insert**: permette di inserire nuovi oggetti (ennuple)
- **update**: permette di modificare il contenuto
- **delete**: permette di eliminare oggetti
- **select**: permette di leggere la risorsa
- **references**: permette la definizione di vincoli di integrità referenziale verso la risorsa (può limitare la possibilità di modificare la risorsa da parte del proprietario, ad esempio con “no action”)
- **alter**: permette modifiche allo schema
- ...

Grant e Revoke

Concessione di privilegi:

- *grant* *<Privileges | all privileges>*
 on Resource
 to Users [with grant option]
- *grant option* specifica se il privilegio può essere trasmesso ad altri utenti

```
grant select on Department to  
Stefano with grant option
```

Grant e Revoke

Revoca di privilegi

- *revoke Privileges on Resource from Users*
[restrict | cascade]
- Restrict: se gli Users hanno concesso i privilegi ad altri, blocca la revoca
- Cascade: se gli Users hanno concesso i privilegi ad altri, propaga la revoca in cascata

Esempi

- GRANT SELECT ON films TO PUBLIC;
- REVOKE INSERT ON films FROM PUBLIC;
(PUBLIC = tutti gli utenti)
- GRANT SELECT, UPDATE ON mytable TO bill;
- GRANT SELECT(title), UPDATE(title) ON films TO alice;
- GRANT ALL PRIVILEGES ON films TO bill;

Segretezza

- La gestione delle autorizzazioni deve “nascondere” gli elementi cui un utente non può accedere, senza sospetti
- Esempio:
 - **Impiegati** non esiste
 - **ImpiegatiSegreti** esiste, ma l'utente non è autorizzato
- L'utente deve ricevere lo stesso messaggio se cerca di accedere alle due tabelle

Visibilità

- Come autorizzare un utente a vedere solo alcune tuple di una relazione?
 - Attraverso una **vista**:
 - Definiamo la vista con una condizione di selezione
 - Attribuiamo le autorizzazioni sulla vista, anziché sulla relazione di base

Autorizzazioni e ruoli

- Concetto di ruolo, cui si associano privilegi (anche articolati), poi concessi agli utenti attribuendo il ruolo
- Attribuisce la membership al ruolo “amministratori” allo user “joe”:
 - GRANT SELECT on films TO amministratori;
 - GRANT amministratori TO joe;

Creazione utenti e ruoli

- Creo un nuovo utente:
 - `CREATE USER jim WITH PASSWORD 'jw8s0F4';`
- Creo un nuovo ruolo:
 - `CREATE ROLE amministratori;`